

# Language Modeling

## Count-Based Models, RNNs, and GPT-Style Models

Yasser Hifny

Faculty of Computers and Artificial Intelligence  
Capital University

# 1. What Is a Language Model?

## Goal

A language model (LM) assigns probabilities to token sequences and predicts the next token from the preceding context.

For a sequence  $w_1, \dots, w_T$ , the model estimates  $P(w_1, \dots, w_T)$ . A next-token LM estimates  $P(w_t \mid w_1, \dots, w_{t-1})$ .

Context  $\longrightarrow$  Probability distribution over next tokens

- Applications: speech recognition, text generation, spelling correction, machine translation, and document understanding.
- We will compare count-based, recurrent, and GPT-style neural approaches.

## 2. Joint Probability and the Chain Rule

The probability of a token sequence can be factorized exactly using the probability chain rule:

$$P(w_1, \dots, w_T) = P(w_1) \prod_{t=2}^T P(w_t \mid w_1, \dots, w_{t-1}).$$

Equivalently,

$$P(w_{1:T}) = \prod_{t=1}^T P(w_t \mid w_{1:t-1}),$$

where the first token is conditioned on an empty context or a beginning-of-sequence symbol.

### Challenge

The full history  $w_{1:t-1}$  can be long, and many distinct histories occur rarely or never in a finite corpus.

### 3. The N-gram Approximation

An  $n$ -gram LM approximates the full history by the preceding  $n - 1$  tokens:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

Thus the joint probability is approximated by

$$P(w_{1:T}) \approx \prod_{t=1}^T P(w_t \mid w_{t-n+1:t-1}).$$

- Unigram: no preceding-token context.
- Bigram: one preceding token.
- Trigram: two preceding tokens.

Use start symbols such as  $\langle s \rangle$  to represent sentence beginnings and an end symbol  $\langle /s \rangle$  when scoring complete sentences.

## 4. Corpus Counts

Given a tokenized training corpus, define:

$C(w)$  = number of occurrences of token  $w$

$C(u, v)$  = number of adjacent occurrences of  $u, v$

$C(u, v, w)$  = number of adjacent occurrences of  $u, v, w$ .

Example corpus:

```
the cat sat
the cat slept
the dog sat
```

The counts estimate empirical frequencies. Consistent tokenization, sentence boundaries, and corpus splits are essential for valid comparisons.

## 5. Unigram Language Model

The unigram model assumes each token is independent of its context:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t).$$

Maximum-likelihood estimation (MLE):

$$\hat{P}_{\text{ML}}(w) = \frac{C(w)}{\sum_{v \in V} C(v)}$$

The sequence probability is

$$\hat{P}(w_{1:T}) = \prod_{t=1}^T \hat{P}(w_t).$$

### Interpretation

The model captures token frequency but ignores word order. It may assign the same probability to sequences containing the same token counts.

## 6. Bigram Language Model

The bigram model conditions each token on the immediately preceding token:

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-1}).$$

MLE estimation:

$$\hat{P}_{\text{ML}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}$$

Joint probability:

$$P(w_{1:T}) \approx P(w_1) \prod_{t=2}^T P(w_t \mid w_{t-1}).$$

Bigram models encode local order and are easy to train, but unseen adjacent pairs receive zero probability under unsmoothed MLE.

## 7. Trigram Language Model

The trigram model uses the previous two tokens:

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-2}, w_{t-1}).$$

MLE estimation:

$$\hat{P}_{\text{ML}}(w_t \mid w_{t-2}, w_{t-1}) = \frac{C(w_{t-2}, w_{t-1}, w_t)}{C(w_{t-2}, w_{t-1})}$$

Approximate sequence factorization:

$$P(w_{1:T}) \approx P(w_1)P(w_2 \mid w_1) \prod_{t=3}^T P(w_t \mid w_{t-2}, w_{t-1}).$$

More context can improve predictions, but the number of possible contexts grows quickly, making sparse counts more common.

## 8. Worked Example: Bigram Counts

Suppose the training corpus contains:

```
<s> the cat sat </s>  
<s> the cat slept </s>  
<s> the dog sat </s>
```

Then  $C(\text{the}) = 3$ ,  $C(\text{the}, \text{cat}) = 2$ , and  $C(\text{the}, \text{dog}) = 1$ .

$$P(\text{cat} \mid \text{the}) = \frac{2}{3}, \quad P(\text{dog} \mid \text{the}) = \frac{1}{3}.$$

But a bigram never seen in training, such as  $(\text{cat}, \text{dog})$ , has MLE probability zero.

## 9. The Zero-Probability Problem

If any token transition in a test sequence has estimated probability zero, the whole product becomes zero:

$$P(w_{1:T}) = \prod_t P(w_t \mid \text{context}) = 0.$$

This is problematic because a held-out corpus naturally contains valid n-grams absent from the training corpus.

### Solution

Smoothing reallocates some probability mass to unseen or rare events. Laplace (add-one) smoothing is a simple baseline.

Other methods include Good–Turing estimation, interpolation, backoff, and Kneser–Ney smoothing.

## 10. Laplace (Add-One) Smoothing

For a context  $h$  and vocabulary  $V$ , add one to every next-token count:

$$P_{\text{Laplace}}(w \mid h) = \frac{C(h, w) + 1}{C(h) + |V|}$$

For bigrams:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + |V|}.$$

For trigrams:

$$P_{\text{Laplace}}(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i) + 1}{C(w_{i-2}, w_{i-1}) + |V|}.$$

## 11. Laplace Smoothing: Numerical Example

Assume  $|V| = 5$ ,  $C(\text{the}) = 3$ , and  $C(\text{the}, \text{cat}) = 2$ . Without smoothing:

$$P(\text{cat} \mid \text{the}) = \frac{2}{3} \approx 0.667.$$

With add-one smoothing:

$$P(\text{cat} \mid \text{the}) = \frac{2 + 1}{3 + 5} = \frac{3}{8} = 0.375.$$

For an unseen continuation:

$$P(\text{bird} \mid \text{the}) = \frac{0 + 1}{3 + 5} = \frac{1}{8} = 0.125.$$

### Trade-off

Laplace smoothing prevents zeros, but for large vocabularies it can over-allocate probability to unseen events and underperform stronger smoothing methods.

## 12. Vocabulary, Unknown Tokens, and Boundaries

- Build the vocabulary from the training split only.
- Map rare or unseen words to <unk> using a documented rule.
- Include sentence-start markers for bigram/trigram contexts and end markers if scoring sentence termination.
- Apply the same text normalization and tokenization to training, validation, and test sets.
- Do not let validation or test text contribute to n-gram counts or vocabulary decisions.

$$V = V_{\text{train}} \cup \{<\text{unk}>, <\text{s}>, </\text{s}>\}$$

The exact conventions affect perplexity and must be reported.

## 13. Cross-Entropy and Perplexity

For a test sequence with  $N$  predicted tokens, average negative log-likelihood is

$$H = -\frac{1}{N} \sum_{t=1}^N \log P(w_t \mid w_{<t}).$$

Perplexity is

$$\text{PPL} = \exp(H) = \exp \left( -\frac{1}{N} \sum_{t=1}^N \log P(w_t \mid w_{<t}) \right)$$

If logarithms base 2 are used, perplexity is  $2^H$ . Lower perplexity indicates that the model assigns higher average probability to the observed test tokens, assuming comparable tokenization and evaluation protocol.

## 14. Perplexity: Intuition and Caveats

- Perplexity can be interpreted as the model's effective uncertainty about the next token.
- A model that assigns high probability to the correct next token receives lower loss and lower perplexity.
- A single zero probability makes unsmoothed sequence NLL infinite.
- Perplexity values are not directly comparable across different tokenizers, vocabularies, or token-counting conventions.
- Always evaluate on held-out data and exclude padding tokens from the loss.

### Example

If average natural-log loss is  $H = 2$ , then  $\text{PPL} = e^2 \approx 7.39$ .

## 15. A Fair LM Evaluation Protocol

- 1 Create train, validation, and test splits before training.
- 2 Fit n-gram counts on training text only; use validation data to select settings.
- 3 Use validation perplexity to choose vocabulary cutoff, order, and smoothing where appropriate.
- 4 Evaluate the final selected models once on the same test split.
- 5 Report tokenization, vocabulary size, unknown-token policy, context length, and number of scored tokens.

Use negative log-likelihood as the primary metric and perplexity as its exponentiated form.

## 16. Why Neural Language Models?

Count-based models store explicit counts for discrete contexts. Their limitations include sparse observations and rapidly growing context tables.

### Neural alternative

Learn distributed representations and a parameterized function that predicts the next token, allowing related contexts to share statistical strength.

A recurrent neural network (RNN) summarizes the preceding sequence in a hidden state:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}).$$

The state is updated token by token and used to predict  $w_{t+1}$ .

## 17. Vanilla RNN Equations

Let  $\mathbf{x}_t$  be the embedding of the current input token and  $\mathbf{h}_{t-1}$  the previous hidden state:

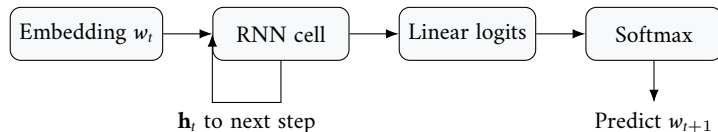
$$\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

The hidden state is projected to vocabulary logits:

$$\mathbf{z}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad P(w_{t+1} = j \mid w_{\leq t}) = \text{softmax}(\mathbf{z}_t)_j.$$

The same recurrent parameters are shared across time steps. Training uses backpropagation through time (BPTT).

## 18. RNN Language Modeling Framework



$$\mathcal{L} = - \sum_{t=1}^{T-1} \log P(w_{t+1} \mid w_{\leq t})$$

At each position, the input prefix predicts the next token; the target sequence is shifted by one token.

## 19. Training an RNN LM: Shifted Targets

Example sequence: the cat sat

|        |  |     |     |     |
|--------|--|-----|-----|-----|
| Input  |  | <s> | the | cat |
| Target |  | the | cat | sat |

For each position, compute vocabulary logits and cross-entropy against the next token:

$$\mathcal{L}_{\text{CE}} = - \sum_t \log P_{\theta}(w_{t+1} \mid w_{\leq t}).$$

- Use batches of token sequences with padding where needed.
- Ignore padded target positions in the loss.
- Clip gradients if recurrent training becomes unstable.

## 20. RNN Strengths and Limitations

### Strengths

- Processes variable-length sequences.
- Shares parameters across time.
- Hidden state summarizes prior context.
- Supports next-token generation.

### Limitations

- Sequential computation limits parallelism.
- Long-range information can be difficult to preserve.
- Vanilla RNNs may suffer vanishing/exploding gradients.
- LSTM/GRU cells help but remain sequential.

## 21. From RNNs to Transformer LMs

GPT-style models use a Transformer decoder stack with causal self-attention.

- Token and positional embeddings represent the input sequence.
- Causal attention allows each position to attend only to itself and earlier positions.
- Feed-forward blocks transform token representations.
- A vocabulary projection produces next-token logits.

$$P(w_{1:T}) = \prod_{t=1}^T P_{\theta}(w_t \mid w_{<t})$$

Like an RNN LM, GPT is autoregressive; unlike an RNN, it can process training positions in parallel using a causal attention mask.

## 22. Causal Self-Attention

For hidden representations  $H$ :

$$Q = HW^Q, \quad K = HW^K, \quad V = HW^V$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} + M \right) V$$

where  $M_{ij} = -\infty$  when key position  $j$  is in the future of query position  $i$ .

### Causal constraint

A token can use information from the left context but must not see future target tokens. This prevents leakage during next-token training.

## 23. GPT-Style Next-Token Objective

Given tokens  $w_1, \dots, w_T$ , GPT predicts each token from its prefix:

$$\mathcal{L}_{\text{LM}}(\theta) = - \sum_{t=1}^T \log P_{\theta}(w_t \mid w_{<t})$$

In a batched implementation, logits at positions  $0, \dots, T-2$  predict labels at positions  $1, \dots, T-1$ .

- Train by minimizing token-level cross-entropy.
- Use attention masks and ignore padding labels.
- Validate with mean loss and perplexity.
- Generate text by repeatedly sampling or selecting the next token.

## 24. GPT Pre-training and Fine-tuning

- ➊ **Pre-training:** learn next-token prediction from a large text corpus.
- ➋ **Fine-tuning:** optionally continue training on domain-specific text such as SANAD.
- ➌ **Evaluation:** measure held-out next-token NLL and perplexity.

$$\text{PPL} = \exp \left( -\frac{1}{N} \sum_{t=1}^N \log P_{\theta}(w_t \mid w_{<t}) \right)$$

For a fair baseline, compare against n-gram models on the same test documents and compatible tokenization/counting conventions. Clearly disclose if a pretrained GPT tokenizer differs from the word tokenizer used for n-grams.

# GPT model

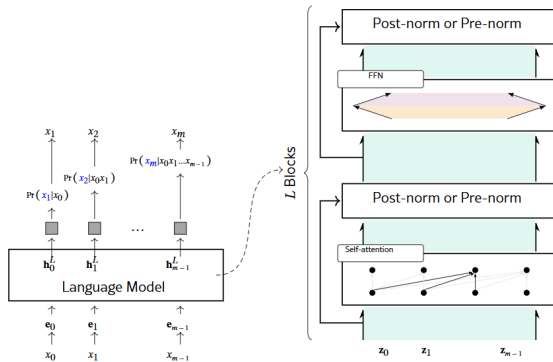


Figure: GPT model from <https://arxiv.org/pdf/2501.09223>.

## 25. Practical GPT Options in PyTorch

A practical assignment can use a pretrained causal language model from Hugging Face Transformers.

- Choose a model with a license and size suitable for the available hardware.
- Use its matching tokenizer and causal-LM model class.
- Split SANAD documents by document, not by overlapping chunks, to avoid leakage.
- Use a validation split for model selection and the test split only for final evaluation.
- If training from scratch, use a small decoder-only Transformer and report its parameter count and training budget.

### Important

SANAD is a document classification dataset, not automatically a language-model benchmark. Treat each document as text for next-token training/evaluation, and prevent the same document from appearing in both training and test splits.

## 26. Assignment: SANAD Language Modeling

### Objective

Implement a count-based language model and a GPT-style neural language model using the SANAD dataset, then compare their test perplexity.

### Part A: Count-based LM

- 1 Tokenize SANAD text consistently and build a training-only vocabulary.
- 2 Implement unigram, bigram, and trigram count estimates.
- 3 Implement add-one smoothing for bigram and trigram models.
- 4 Select the n-gram order/smoothing setting using validation perplexity.

### Part B: GPT-style LM

- 1 Fine-tune a pretrained causal LM or train a small GPT-style decoder.
- 2 Compute next-token loss and perplexity on held-out SANAD text.

## 27. Assignment: Experimental Protocol

- 1 Make reproducible train/validation/test splits at document level.
- 2 Record the SANAD subset/category coverage and the number of documents and tokens.
- 3 Keep normalization policy documented and apply it consistently.
- 4 For n-grams, state  $n$ , vocabulary size, unknown-token handling, and smoothing.
- 5 For GPT, state model name, tokenizer, maximum sequence length, stride/chunking, training epochs, and hardware.
- 6 Ensure test documents never contribute to training, vocabulary fitting, or model selection.

## 28. Assignment: Perplexity Comparison

Report the following table on the held-out test set:

| Model             | Tokenizer       | Test tokens | Perplexity |
|-------------------|-----------------|-------------|------------|
| Unigram MLE       | Word            | ...         | ...        |
| Bigram + add-one  | Word            | ...         | ...        |
| Trigram + add-one | Word            | ...         | ...        |
| GPT-style LM      | Model tokenizer | ...         | ...        |

### Interpretation

Lower perplexity is better only when the tokenization and scoring conventions are comparable. If tokenizers differ, report the results transparently and avoid claiming a strict apples-to-apples ranking.

## 29. Assignment: Deliverables and Rubric

### Deliverables

- Reproducible PyTorch/Python source code and run instructions.
- Count tables or compact count-model implementation.
- Training/validation loss curves for the neural model.
- Test perplexity table, token counts, and model/configuration details.
- Short discussion of sparsity, smoothing, context length, and compute cost.

### Suggested rubric

- Correct factorization and count estimation: 25%
- Correct smoothing and perplexity calculation: 20%
- GPT training/evaluation pipeline: 30%
- Fair experiment and leakage prevention: 15%
- Analysis and reproducibility: 10%

## 30. Common Mistakes to Avoid

- Computing vocabulary or n-gram counts from validation/test data.
- Reporting perplexity without stating the tokenization.
- Forgetting to shift GPT inputs and labels by one token.
- Including padding positions in the loss or token count.
- Allowing future-token leakage in a Transformer attention mask.
- Treating overlapping chunks from one document as independent splits.
- Comparing models with different test text or inconsistent sentence-boundary rules.

## 31. Summary

- The chain rule factorizes a sequence probability into next-token conditional probabilities.
- N-gram LMs approximate context with the preceding  $n - 1$  tokens.
- MLE estimates unigram, bigram, and trigram probabilities from counts.
- Laplace smoothing assigns nonzero probability to unseen events.
- Perplexity is the exponential of average negative log-likelihood.
- RNNs use a recurrent hidden state; GPT-style models use causal self-attention.
- The SANAD assignment compares count-based and GPT-style next-token prediction under a controlled evaluation protocol.

## Questions?

Count-based LMs  $\longrightarrow$  RNN LMs  $\longrightarrow$  GPT-style LMs